Active learning route and feedback

Blue Impulse Study Guide V1.0 — Stage 2 complete edition, 7 November 2026

This complete edition preserves the original cover, chapter dividers and developed teaching pages, then adds the active learning missions, optional resource introductions and explained exercise answers. The story follows Pip, a fictional campus delivery rover; robot-7 is its identifier in the packet exercise. Fictional model measurements are labelled and are not research results.

How to use the complete guide

Start with the learning goals and core explanations. In every chapter, pause at the exercise and make a prediction before reading feedback. Use the matching chapter mission below to apply the ideas, record the result and explain why it happens. Then compare the expected answer and try a changed input. The answer key is placed after all six missions so learners can attempt the questions first.

Prerequisites are basic arithmetic and the earlier chapters in sequence. Paper and a calculator are sufficient. Optional code is supplied in the submission's supporting folder and needs Python 3.9 or newer, with no extra packages or account. The separate editable Test Bank provides 54 balanced questions for review; its questions come before its answer key. Human peer review remains pending, as documented in the package's review record.

Complete edition navigation

The original six-chapter contents and bookmarks remain usable. Additional PDF bookmarks open each mission, resource introduction and explained-answer section. The teaching topics remain named consistently across Markdown, slides and videos; revised videos cite chapter number, chapter name and subtopic rather than page numbers.

Mission 1 — The parcel total is wrong

ACT-01 — The parcel total is wrong

Chapter 1 — The Fundamentals

Pip's delivery receipt says 200 parcels. The team expected 7. A programmer has confused an address with a value. Can you find the mistake?

Goal: Trace stored instructions and distinguish an immediate value, an address and the content stored there.

Preparation: Paper and pencil are sufficient. Optional: Python 3.9 or newer and supporting/BlueImpulse_Demonstrations.py. This is an invented teaching ISA, not a real processor.

Task:

1. Write columns for PC before fetch, instruction, ACC after execution and memory[200]. Start at PC 100, ACC 0 and memory[200] 0.

2. Run: 100 LOAD #3; 101 ADD #4; 102 STORE 200; 103 HALT. The # symbol means a literal value. PC advances by one when each instruction is fetched.

3. Predict the final result before checking the output. Then replace ADD #4 with ADD #9 and trace again.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic execution --add 9. Compare its state trace with your table.

Observe and record: Record ACC after every instruction, memory[200] after STORE, and final PC. Note the difference between a number used as data and a number used as an address.

Reflect: Why does STORE 200 copy 7 rather than replace the accumulator with 200? Which parts of the model would depend on a real ISA?

Feedback — check after trying: First run: after LOAD, ACC 3; after ADD, ACC 7; after STORE, memory[200] 7; after HALT, final PC 104. The modified run stores 12. STORE's operand is a destination address. PC increments, instruction size and register names are model choices, not universal CPU rules.

Optional resource: R2 (full introduction and offline alternative follow).

R2 — Nand to Tetris — Project 5: Computer Architecture

Noam Nisan and Shimon Schocken / Nand to Tetris.

https://www.nand2tetris.org/project05

Relevant section: Project overview, proposed computer architecture and hardware simulator guidance

Why use it: A concrete educational computer architecture connects logic, CPU state and memory.

Learner task: After ACT-01, compare the purpose of an instruction address and a data-memory address with the project's computer model. Name one part the models share and one detail that differs.

Limits: The Hack computer is a different ISA; its instructions and registers must not be substituted directly into this guide's toy program. Software installation is optional and not required for the paper task.

Offline alternative: ACT-01 includes its full program, trace and expected output. ACT-03 provides a complete paper pipeline grid specification.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 2 — One lucky route is not a policy

ACT-02 — One lucky route is not a policy

Chapter 2 — Behaviorism & Behaviorist AI

Pip takes a new route and arrives early once. "Best route forever!" it announces. Its teammate asks whether one reward is enough evidence.

Goal: Calculate a Q-learning update and distinguish an estimate from a guaranteed outcome.

Preparation: Calculator or paper. Optional standard-library Python script. Know reward, policy, value, alpha and gamma from this chapter.

Task:

1. Use old Q = 2, alpha = 0.5, reward = 1, gamma = 0.9 and best next Q = 4. Predict whether the estimate will rise or fall.

2. Calculate target = reward + gamma × best next Q; error = target − old Q; change = alpha × error; new Q = old Q + change.

3. Repeat with the same old Q, alpha and reward but a terminal next state. Set future value to zero.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic rl. Compare values. Explain whether either update proves a route is safest.

Observe and record: Record the target, temporal-difference error, change and new value for both cases.

Reflect: Why is the future term zero at termination? Why can exploration remain useful after a positive outcome? What safety rule would constrain a real robot's exploration?

Feedback — check after trying: Non-terminal: target 4.6, error 2.6, change 1.3, new Q 3.3. Terminal: target 1, error −1, change −0.5, new Q 1.5. These are updated estimates, not proofs. Use many relevant transitions and evaluate behaviour against the task, including safety constraints.

Optional resource: R1 (full introduction and offline alternative follow).

R1 — Reinforcement Learning: An Introduction, second edition

Richard S. Sutton and Andrew G. Barto, MIT Press, 2018.

http://incompleteideas.net/book/the-book-2nd.html

Relevant section: Chapter 3 (finite Markov decision processes) and Section 6.5 (Q-learning)

Why use it: Primary textbook support for the agent/environment distinction, expected return and the Q-learning update.

Learner task: After ACT-02, explain in your own words what the bootstrapped target includes, then compare it with the terminal-state case.

Limits: The book is mathematically deeper than this beginner guide. Start with the supplied worked calculation; do not treat the browser page as required activity content.

Offline alternative: The formula, both complete calculations and interpretation are included in ACT-02 and the chapter, so the task works offline.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 3 — The kitchen with four stations

ACT-03 — The kitchen with four stations

Chapter 3 — Hardware

Pip needs three sensor calculations. One worker finishes all four steps of a job before starting the next. Another team proposes four stations. Will each job suddenly take one step?

Goal: Distinguish pipeline throughput from the latency of one instruction and identify the effect of a dependency.

Preparation: Draw a grid with cycles 1–8 as columns and instructions I1–I3 as rows. Use stages F (fetch), D (decode), E (execute), W (write back). Assume one cycle per stage, independent instructions and no stalls initially.

Task:

1. Schedule I1 in F/D/E/W across cycles 1–4; start I2 at cycle 2 and I3 at cycle 3.

2. Predict the finish cycles and compare total elapsed cycles with three instructions run without overlap.

3. Now assume I2 needs a result from I1 that is unavailable until the end of I1's W. State this dependency explicitly; postpone I2's E to cycle 5 and propagate later stages without letting instructions overtake.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic hardware. Use its ideal trace for the first case. Treat your stalled case as a separate scheduling model.

Observe and record: Ideal finish cycles are 4, 5 and 6. Mark every occupied stage and every inserted wait. Do not equate elapsed cycles with the sum of all stage work.

Reflect: What improved: single-instruction latency or completed instructions per unit time? Why must a claim of speedup list its assumptions?

Feedback — check after trying: Ideal case takes six cycles rather than twelve sequential stage cycles, but each instruction still uses four stages. In the stated dependency model I2 executes at 5 and writes at 6; I3 must wait behind it and can execute at 6 and write at 7. Real forwarding, stage timings and hazards differ, so this trace does not predict every CPU.

Optional resource: R2 (full introduction and offline alternative follow).

R2 — Nand to Tetris — Project 5: Computer Architecture

Noam Nisan and Shimon Schocken / Nand to Tetris.

https://www.nand2tetris.org/project05

Relevant section: Project overview, proposed computer architecture and hardware simulator guidance

Why use it: A concrete educational computer architecture connects logic, CPU state and memory.

Learner task: After ACT-01, compare the purpose of an instruction address and a data-memory address with the project's computer model. Name one part the models share and one detail that differs.

Limits: The Hack computer is a different ISA; its instructions and registers must not be substituted directly into this guide's toy program. Software installation is optional and not required for the paper task.

Offline alternative: ACT-01 includes its full program, trace and expected output. ACT-03 provides a complete paper pipeline grid specification.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 4 — The fast network sends a late warning

ACT-04 — The fast network sends a late warning

Chapter 4 — Networking

Pip's dashboard celebrates a high link rate. A stop warning arrives half a second late. "But the Wi-Fi is fast!" says the operator. The parcels keep moving.

Goal: Separate link capacity from response latency and calculate distance travelled during communication delay.

Preparation: Paper or calculator. Use a constant illustrative speed of 30 m/s and delays 500, 50, 5 and 200 milliseconds. This is not a campus-rover speed claim or a braking simulation.

Task:

1. Predict which delay causes the greatest extra travel before a response.

2. Convert each delay to seconds by dividing by 1,000. Calculate delay distance = speed × delay.

3. Create a two-column table of delay and distance. Explain why increasing Mb/s alone need not remove propagation, queueing or processing delay.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic network. Compare results and change the speed in your own calculation to 2 m/s.

Observe and record: Record units at each step and compare ratios: a ten-times smaller delay creates ten-times smaller delay distance at constant speed.

Reflect: What does the calculation omit? How could jitter or occasional long queueing delays matter even when average latency is low?

Feedback — check after trying: 500/50/5/200 ms correspond to 0.5/0.05/0.005/0.2 s and 15/1.5/0.15/6 m at 30 m/s. At 2 m/s they give 1/0.1/0.01/0.4 m. This excludes reaction after receipt, actuator lag and braking distance; do not call it total stopping distance.

Optional resource: R3 (full introduction and offline alternative follow).

R3 — RFC 1122 — Requirements for Internet Hosts: Communication Layers

IETF / R. Braden, editor, 1989.

https://www.rfc-editor.org/rfc/rfc1122

Relevant section: Section 1.1.3 (Internet protocol suite) and the link, IP and transport sections

Why use it: An authoritative foundation for the layered Internet model used in Chapter 4.

Learner task: Draw the four teaching layers and place HTTP, TCP, IP and Ethernet/Wi-Fi by role. Explain why a large link capacity does not set an end-to-end response time.

Limits: This is a technical historical standards document; current protocols have later updates. Use it for layer roles, not as a complete modern security checklist.

Offline alternative: The chapter and ACT-04 contain the layer explanations and all delay inputs/results offline.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 5 — Yesterday's command arrives today

ACT-05 — Yesterday's command arrives today

Chapter 5 — Applications

Pip has stopped safely. A delayed "move" packet arrives and looks correctly formatted. Should it obey a message just because the bytes make sense?

Goal: Apply identity, unit, version, freshness and sequence checks while distinguishing secure transport from safe command handling.

Preparation: Receiver device ID robot-7; expected units m; policy_version 3; maximum age 200 ms; last accepted sequence 41. Assume the upstream authentication check is true for these examples. The toy Boolean is not a cryptographic implementation.

Task:

1. Inspect packets in order: A = sequence 42, age 50 ms; B = sequence 43, age 600 ms; C = sequence 42, age 60 ms. All other listed fields match.

2. For each packet, decide ACCEPT, REJECT STALE or REJECT DUPLICATE. Only update last accepted sequence when a packet is accepted.

3. Predict what would happen if a developer updated last sequence before checking age, and explain why reject paths should not corrupt accepted-state bookkeeping.

4. Optional command: python3 supporting/BlueImpulse_Demonstrations.py --topic networked-ai. Test the same order and compare receiver state.

Observe and record: Write the decision and last accepted sequence after each packet. Identify which check fails; do not invent a new packet timestamp.

Reflect: What must a real deployment add for clock assumptions, authentication, session resets and a safe local response when the network fails?

Feedback — check after trying: A is ACCEPT and advances last sequence to 42. B is STALE; last sequence stays 42. C is DUPLICATE; last sequence stays 42. Version, identity and unit checks add semantic protection. Real systems need authenticated messages, replay/session design, trustworthy timing and explicit fallback, not only this toy validation function.

Optional resource: R4 (full introduction and offline alternative follow).

R4 — RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3

IETF / Eric Rescorla, 2018.

https://www.rfc-editor.org/rfc/rfc8446

Relevant section: Section 1 (Introduction) and Section 8 (0-RTT and anti-replay)

Why use it: Distinguishes transport confidentiality/integrity from application-level decisions, including replay considerations.

Learner task: After ACT-05, explain why authenticated transport alone does not decide whether an old command is still appropriate to execute.

Limits: The Python receiver is a validation demonstration, not a TLS implementation. Do not implement cryptography from this worksheet.

Offline alternative: ACT-05 includes the packet fields, state transitions and rejection explanations; no network service or account is needed.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Mission 6 — Choose a system, not a leaderboard

ACT-06 — Choose a system, not a leaderboard

Chapter 6 — Strengths & Limitations

A larger model wins an accuracy contest but overheats Pip's computer. A cloud model is accurate too, until the network drops. The team must choose a deployment, not a single impressive number.

Goal: Evaluate edge/cloud placement against explicit latency, memory, power, privacy and resilience constraints.

Preparation: Use this fictional design table: local small model = 30 ms compute, 180 MB working memory, 4 W, 92% measured task accuracy; local large model = 90 ms, 700 MB, 9 W, 96%; remote model = 20 ms server compute plus 100–300 ms round trip, 98%, uploads images. Device budgets: 512 MB, 6 W and 80 ms deadline. Treat accuracies as supplied hypothetical measurements, not research results.

Task:

1. Eliminate designs that violate hard local memory, power or timing budgets. For the remote model add the network and server times.

2. Choose where emergency control and overnight fleet analytics should run. State how the system behaves during lost connectivity.

3. A software update improves the small model to 94% but needs 70 ms and 5 W. Decide what to test before accepting it; do not assume that one benchmark settles safety.

4. Draw a two-column decision record: constraint/evidence, then choice/trade-off. Propose one privacy safeguard and one monitoring measure.

Observe and record: Separate hard failures from preferences and record the worst stated time rather than only the mean. Identify unknowns such as warm-device timing and test-data coverage.

Reflect: Why is 98% accuracy not sufficient to choose the remote model for the 80 ms task? Which human decision remains after the calculations?

Feedback — check after trying: The small local model fits the stated budgets. The large local model violates memory, power and latency. The remote option takes 120–320 ms and misses the deadline; it may suit slower analytics with privacy controls. The updated small model fits the listed limits but leaves only 10 ms timing margin, requiring end-to-end/warm-device testing, relevant accuracy evaluation and a fallback. Valid alternatives earn credit if their assumptions and trade-offs are explicit.

Optional resource: R5 (full introduction and offline alternative follow).

R5 — Artificial Intelligence Risk Management Framework (AI RMF 1.0)

NIST, NIST AI 100-1, January 2023.

https://www.nist.gov/itl/ai-risk-management-framework

Relevant section: Framework functions Govern, Map, Measure and Manage; trustworthy AI characteristics

Why use it: Connects measured performance with context, impacts, monitoring and responsibility.

Learner task: For ACT-06, assign one concrete action to each framework function: identify an owner, describe the operating domain, measure a relevant failure and plan a response.

Limits: A voluntary risk-management framework is not certification that a product is safe, fair or legally compliant. Context and evidence still matter.

Offline alternative: ACT-06 supplies the design table and expected trade-off reasoning. A suitable four-action response is: Govern — name an accountable operator; Map — define the campus operating domain; Measure — test missed deadlines and uneven accuracy; Manage — maintain fallback and an incident response process.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Exercise answer key

Attempt the original guide exercises first. Match each question number with the explained answer below. Equivalent correct explanations and clearly stated alternative assumptions are acceptable.

Chapter 1 — The Fundamentals

Answer 1.1a1

79. The sign bit is 0, so add the weights 64 + 8 + 4 + 2 + 1.

Answer 1.1a2

−102. In eight-bit two's complement, −128 + 16 + 8 + 2 = −102. Alternatively invert 10011010 and add 1 to obtain 01100110 = 102, then negate.

Answer 1.1a3

11001011. +53 is 00110101; invert to 11001010 and add 1. Check: −128 + 64 + 8 + 2 + 1 = −53.

Answer 1.1a4

−128 through +127 inclusive: −2^7 to 2^7 − 1. There are 256 distinct eight-bit patterns.

Answer 1.1b1

0111 1001 1000. Encode the decimal digits 7, 9 and 8 separately; do not convert the entire value to ordinary binary.

Answer 1.1b2

9776. The four nibbles represent decimal 9, 7, 7 and 6.

Answer 1.1c

A decimal digital display can store each displayed digit as one BCD nibble. Some decimal arithmetic systems also use BCD; it uses more bits than ordinary binary for most multi-digit values.

Answer 1.2a1

Sampling resolution (bit depth) is the number of bits used to encode the amplitude of each audio sample.

Answer 1.2a2

More bits per sample provide more quantisation levels, reducing amplitude quantisation error for a given signal range. This increases raw storage and does not itself increase the number of time samples per second.

Answer 1.2b1

For a stored bitmap, pixel dimensions describe the number of pixels across and down. In display or print contexts, resolution can also mean pixel density. State the intended meaning and units.

Answer 1.2b2

4 bits per pixel, because 2^4 = 16 colour codes.

Answer 1.2b3

256 colours require 8 bits = 1 byte per pixel. 16,384 × 512 = 8,388,608 bytes. Divide by 1,073,741,824 bytes/GiB to obtain 0.0078125 GiB (8 MiB). This excludes palette, header and compression.

Answer 1.3

PC → MAR; memory[MAR] → MDR; increment PC; MDR → CIR; decode; execute. This is the supplied one-word teaching model. Implementations can overlap operations and use different instruction lengths.

Answer 1.4

A backup permits recovery after accidental deletion, corruption, device failure or an attack. Keep a separate recoverable copy with appropriate access controls and test restoration; synchronisation alone can also synchronise a deletion.

Answer 1.5

An assembler translates assembly operations and labels into machine instructions. A compiler translates source into another representation, such as native instructions or bytecode. An interpreter executes operations through a runtime. Real implementations can combine these approaches; a language is not inherently limited to one.

Chapter 2 — Behaviorism & Behaviorist AI

Answer 2.1

Classical conditioning associates a cue with another stimulus so the cue can elicit a response. Operant conditioning changes the likelihood of an action through its consequences. A bell paired with food and a voluntary action followed by a reward illustrate the difference. They are analogies, not proof that numerical RL reproduces animal experience.

Answer 2.2

A thermostat is a fixed reflex example: environment = room; stimulus = measured temperature below the set point; response = switch heating on. With unchanged thresholds it reacts but does not learn a policy from rewards.

Answer 2.3

Subsumption coordinates behaviour modules with defined priority, inhibition or suppression relationships. Obstacle avoidance may suppress a wandering command. Useful behaviour can emerge without a single detailed central map, although practical robots can combine reactive modules with mapping and planning.

Answer 2.4a

The agent chooses an action using its policy; the environment produces the next state and reward according to the task's dynamics. An MDP specifies states, actions, transition probabilities, rewards and discounting. A policy maps states to actions or action probabilities; it is not the reward function.

Answer 2.4b

Exploration obtains information about insufficiently tested actions. Exploitation chooses an action using the best current value estimate, which may include future rewards. Always exploiting can lock in an inferior early estimate; always exploring can waste known good actions. Physical exploration needs safety constraints.

Chapter 3 — Hardware

Answer 3.1a

Primary storage includes working memory and, in some classifications, ROM; it is accessed through the processor's memory system. Main RAM is normally volatile and relatively fast. Secondary storage such as SSD or HDD is non-volatile and usually slower to access. Data normally moves into working memory before ordinary processing. "All primary storage is volatile" is false.

Answer 3.1b

Magnetic: hard disk drive or magnetic tape. Optical: CD, DVD or Blu-ray. Solid-state: SSD or USB flash drive.

Answer 3.2a

SRAM uses a bistable transistor cell and does not need periodic refresh while powered. DRAM stores charge that leaks and needs refresh. Both normally lose data without power. SRAM is fast but less dense and costlier per bit; DRAM supports larger main memories.

Answer 3.2b

Mask ROM is fixed during manufacture. PROM can normally be programmed once. EPROM can be erased with ultraviolet light and reprogrammed; EEPROM and flash instead use electrical erasure. These names refer to different technologies, not one universal write process.

Answer 3.3

Touchscreen = output display plus touch input. SSD = storage, with read/write data transfers. Barcode reader = input. A device can legitimately have more than one role.

Answer 3.4

For a greenhouse, a temperature sensor provides readings to a controller; when the temperature exceeds a defined threshold, the controller drives a fan actuator. Measure again to close the loop. A lower switch-off threshold can prevent rapid on/off chatter. This is fixed-rule control unless a learning method actually updates the decision policy.

Answer 3.5a

RISC families traditionally emphasise relatively regular, simple instruction encodings and load/store operations. CISC families traditionally provide more varied encodings and operations. Modern implementations blur boundaries and may translate complex instructions into micro-operations. Neither label alone proves speed or efficiency.

Answer 3.5b

Pipelining overlaps stages of different instructions to increase throughput. In an ideal four-stage pipeline, three independent instructions need 4 + 3 − 1 = 6 cycles, rather than 12 sequential stage cycles. A data dependency, resource conflict or branch can cause stalls; one instruction still traverses all four stages.

Answer 3.5c

PC holds the next instruction address; ACC holds an intermediate arithmetic result in the teaching model; MDR holds a value or instruction being transferred between memory and the processor. A store address belongs in MAR, not automatically in ACC.

Answer 3.6

A hardware interrupt is a request from hardware for service. When an enabled request is accepted, the system saves the execution state it needs, transfers control to an interrupt service routine, then restores state and resumes. Masking, priorities and architecture affect when this occurs.

Answer 3.7

AND: 00→0, 01→0, 10→0, 11→1. OR: 00→0, 01→1, 10→1, 11→1. NOT: 0→1, 1→0. The usual AND symbol has a flat input side and rounded output side; OR has a curved input side; NOT is a triangle with an inversion circle. Named rectangular gate symbols are also valid if used consistently.

Chapter 4 — Networking

Answer 4.1a

A LAN connects a local site, often under one organisation's control. A WAN connects distant sites across wider geography and may depend on carriers. WAN paths often introduce longer propagation paths, but equipment, traffic and access links determine actual throughput and delay; scope alone does not guarantee a speed.

Answer 4.1b

Client–server centralises a service, making policy and backup management easier but creating capacity and availability responsibilities. Peers can both request and provide resources, spreading work but complicating discovery, trust and consistency. Both can scale with suitable designs; neither is automatically unlimited.

Answer 4.1c

A thick client performs substantial local processing and may keep local data. A thin client depends more on a remote server for computation and storage. Thin clients simplify some central management but depend more on network availability; actual systems can be hybrids.

Answer 4.1d

A public cloud uses provider-operated shared infrastructure, often lowering up-front costs but adding provider dependence and shared security responsibilities. A private cloud is dedicated to one organisation and offers more direct control, but the organisation must fund and operate or contract capacity and security. Neither is inherently secure merely because of its label.

Answer 4.2

Legacy shared, half-duplex Ethernet uses CSMA/CD to detect a collision and retry after backoff. Wi-Fi uses collision avoidance, including channel sensing, waiting and acknowledgements. Modern switched full-duplex Ethernet links normally avoid shared-medium collisions; congestion and queues can still occur.

Answer 4.3a

In https://example.org/guide/chapter1, https is the scheme, example.org is the host name and /guide/chapter1 is the path. A URL can also include a port, query and fragment. It identifies how and where to request a resource, not necessarily a fixed physical machine.

Answer 4.3b

DNS resolves names through a distributed hierarchy and caches. A resolver may return an address record for the requested host; the application can then use that address to contact a server. DNS does not itself download the web page, and cached records have limited lifetimes.

Answer 4.4a

Application: services and application messages. Transport: communication between application endpoints, such as TCP streams or UDP datagrams. Internet: IP addressing and routing across networks. Link/network-access: delivery over a local link, such as Ethernet or Wi-Fi. Layer names vary slightly between teaching models.

Answer 4.4b

HTTP requests and transfers web resources. FTP transfers files but its traditional form lacks confidentiality. POP3 retrieves email, often for local storage. IMAP manages and synchronises email kept on a server. Secure variants or TLS are required where credentials and content need protection.

Answer 4.5a

Circuit switching reserves communication resources along a path for a session. Capacity can remain allocated even when no useful data is being sent, giving a different sharing trade-off from statistical packet multiplexing.

Answer 4.5b

Under the stated reserved-capacity requirement, a circuit can provide predictable allocated capacity without competing for that same reservation during the call. It may waste idle capacity and needs setup. Modern Internet video calls normally use packet networks; the exercise is a hypothetical trade-off, not a claim about usual Internet practice.

Answer 4.5c

Benefits: bursty users can share link capacity efficiently, and packets can be forwarded along available paths. Drawback: queueing, loss or reordering can create variable delay and require handling. No route change or reliability is guaranteed merely by using packets.

Answer 4.6a

Security protects systems and data against unauthorised access or harm. Privacy concerns appropriate collection and use of personal information. Integrity concerns accuracy and resistance to unauthorised alteration. Encrypted personal data can still be collected without a legitimate purpose, so confidentiality alone does not settle privacy.

Answer 4.6b

Validation checks whether data satisfies rules such as format or range; a plausible wrong date can pass. Verification checks agreement with an original or expected copy, such as double entry or a transfer checksum. A checksum detects some changes but does not establish authenticity against an active attacker.

Answer 4.6c

Examples: least-privilege access controls, timely security updates, strong authentication, firewalls, encryption, protected backups and malware detection. Explain the threat addressed; no single mechanism stops every threat.

Answer 4.7a

Symmetric cryptography uses a shared secret and is efficient for bulk data, but securely distributing and protecting that secret matters. Asymmetric mechanisms use a public/private key pair for particular tasks such as key establishment or signatures. Both require correct algorithms and key handling; a public key is not itself secret.

Answer 4.7b

The sender signs with a private signing key; recipients verify with the matching public key. A valid signature supports origin and integrity relative to trust in that key, but does not hide the message. Encrypting for confidentiality is a distinct operation. A copied valid old signed message may still require replay protection.

Answer 4.7c

SSL is the obsolete predecessor to TLS. TLS is used today for protected transport in HTTPS and other application protocols, providing confidentiality and integrity with authentication according to configuration. TLS does not make an endpoint safe or prevent a legitimate user from misusing received data.

Chapter 5 — Applications

Answer 5.1

A smart system connects sensing, computation, communication and action to a goal. For irrigation, a moisture sensor supplies readings, software compares them with the policy, a network reports status and a valve changes water flow. It need not use machine learning simply because it is connected.

Answer 5.2a

Real-time processing means meeting a defined timing deadline, not merely being fast on average. Communication, queueing, computation and actuator delays all contribute. At 30 m/s, an extra 0.2 s delay means 6 m travelled before a response; this is delay distance, not total braking distance.

Answer 5.2b

Automation reduces direct manual operation of a task. It can improve consistency and throughput, but can also repeat a wrong rule at scale, fail on unusual inputs or change workers' monitoring responsibilities. Assess the actual process rather than assuming all errors disappear.

Answer 5.3a

An RL algorithm uses transitions and rewards to update policy parameters or value estimates, aiming at expected cumulative reward. A deployed agent may use a fixed learned policy without updating it on every action. Correctly associate feedback with the state and action that produced it.

Answer 5.3b

An actuator turns an electrical control signal into physical action: a motor moves a wheel, a valve changes flow, or a heater produces heat. The controller must respect limits and use feedback where needed; a software command is not proof the requested movement occurred.

Answer 5.4a

Human-AI interaction concerns how people work with AI-driven systems. Uncertainty, opaque decisions, changing performance and delegated actions add challenges to general interface design. Interfaces should communicate limits, support correction and provide meaningful human control.

Answer 5.4b

A user may over-trust an unexplained "safe" decision and ignore a hazard, or be unable to recover when an autonomous action is wrong. Display relevant status and uncertainty, explain the available action and offer an effective override or recovery process.

Answer 5.5a

Ethical AI considers consequences for people as well as performance: fairness across groups, explanations appropriate to users, privacy and accountable responsibility. Trade-offs must be evaluated in context, with monitoring and redress. A fairness score alone is not a complete ethical judgement.

Answer 5.5b

Opportunity: remote monitoring can find faults and reduce wasted energy. Risk: compromised devices can expose personal data or accept dangerous commands. Minimise collected data, authenticate access and maintain update and incident-response processes.

Chapter 6 — Strengths & Limitations

Answer 6.1

An embedded device has limited memory for weights and working data, compute time for deadlines, power from its supply and thermal capacity to remove heat. A model that fits storage can still exceed RAM or latency budgets. Consider quantisation or a smaller model, then measure accuracy and worst-case timing on the target device.

Answer 6.2

Scalability means accommodating growth while meeting acceptable service targets. Interoperability means exchanging and correctly using information between systems. A network that can carry many packets still fails interoperability if one sensor sends centimetres and the receiver assumes metres. Test load, schemas, units, versions and identities.

Answer 6.3

Adaptive intelligence changes behaviour appropriately using new evidence or experience. It can respond to changed conditions, but noisy feedback, distribution shifts or unsafe exploration can make adaptation harmful. Test changes against objectives and retain monitoring and recovery controls.

Answer 6.4

Predictive analytics estimates future or unknown outcomes from evidence, such as a likely equipment failure. Behavioural modelling represents actions and interactions, such as how drivers respond to signs. A model can be useful without proving causes; both require relevant data and evaluation on new cases.

Answer 6.5

Resource optimisation selects where and how to compute while balancing latency, accuracy, energy, bandwidth, privacy and cost. Local emergency control can avoid a remote round trip; cloud processing can support large-scale training or reporting. Edge processing still has hardware limits and requires maintenance.

Answer 6.6

Explainability makes relevant aspects of model behaviour understandable to a particular audience. Deep RL can be difficult to explain because many learned parameters and sequential consequences interact. An approximate post-hoc explanation may be useful, but is not a guarantee of the model's true causal reasoning.

Answer 6.7

Security concerns include unauthorised commands and replay or manipulation of sensor messages. Ethical concerns include unfair impacts, privacy loss and unclear responsibility for harm. Combine authentication, freshness checks, monitoring, human oversight and explicit accountability. Valid alternative risks and controls should earn credit when well justified.

ROVERLAB — Build, Connect, Train

Interactive companion: Play ROVERLAB

Provider: Blue Impulse. **Time:** approximately 5–10 minutes per experiment. **Preparation:** read Chapter 2 — Behaviorism & Behaviorist AI, Chapter 3 — Hardware, and Chapter 4 — Networking; use Chapter 5 — Applications to connect the systems. No programming experience or account is required.

Your challenge

Build an exploration rover that can sense obstacles, visit a research site, send its findings to base and return. Predict how one changed component or response will affect the mission, then compare the prediction with the event log and results.

1. Choose Exploration rover. In Hardware, inspect the processor, memory, motors, wheels or tracks, sensors and power supply. You may try the starter build, then inspect what each part contributes. Remove the distance sensor once: read why sensing or mission readiness changes, then restore it.

2. Add a radio module and connect the base station. Run the network test. Explain why a connected base station is a different role from the robot's processor.

3. Train at least three rules: obstacle detected → stop, then turn; destination reached → return to base; low battery → return to base. Test a stimulus and give feedback. Compare competing associations by strength and order. The research sensor collects its reading at the site and queues it for transmission through your actual network route; the destination rule tells the drive system to return.

4. Select Survey & return. Resolve the readiness findings, then launch. Pause or step through an obstacle event. Identify the sensor observation, selected rule, actuator action and later packet transmission.

5. Record the result and four scores. Return to training and replace obstacle → stop, then turn with obstacle → move / run. Predict the outcome before launching again. Compare the actual event evidence rather than only the total score.

Observe, explain and extend

Record two rows: configuration, prediction, observed event, result, and explanation. Which component detected the obstacle? Which rule selected the response? What made the wheels move? Which route delivered the report? Why can a compatible rule still be unhelpful?

Expected reasoning: an appropriate sensor makes the obstacle observable; the processor selects a supported rule; the drive system carries out the command. Continuing toward the obstacle can cause a collision. A module alone does not establish a usable route, and a route that works at the start may fail as the robot moves. Feedback changes which competing association wins; it does not make an unsafe command safe or supply missing hardware.

For another experiment, use Search & assist to combine person sensing and a control computer, or Monitor & protect to detect high temperature, stop the process and send a server alert. Chapter 6 — Strengths & Limitations helps explain why a successful toy mission is insufficient evidence for a real autonomous deployment.

Access and offline alternative

Open the link in a modern desktop or mobile browser. After a complete first visit, the local game cache supports offline reloads where service workers are available. Progress is saved in this browser; private browsing or blocked storage can limit persistence. The repository also provides a source ZIP that can be extracted and opened at public/index.html without internet:
https://github.com/user-github-me/cs-ai-project-01/tree/main/apps/roverlab

If the game cannot load, draw this trace on paper: obstacle → distance sensor → processor selects rule → motor response → research reading → radio link → base. Compare stop-and-turn with continuing forward; remove the sensor or radio and mark the exact missing link. This supplies the same core reasoning task without the website.

ROVERLAB uses explicit player-authored stimulus–response associations and simplified feedback. It is not a neural network or a full reinforcement-learning trainer. Range, power and environmental values are fictional teaching values. The source and automated checks are public; a student or teacher playtest remains separate evidence.

Sources, access and revision notes

The six team-supplied chapter PDFs defined the scope. Their notes credit Cambridge International AS & A Level Computer Science but do not identify a full edition or author bibliography; this remains a provenance limitation and no edition has been invented. The new activities, calculations and checkpoints are original AI-assisted teaching examples. The guide's original artwork is user supplied and retains its own third-party rights.

The resources below were inspected as optional extensions and reference support. No external link replaces an explanation, input, question or answer in this snapshot. No student-created website or external platform is required.

R1 — Reinforcement Learning: An Introduction, second edition

Richard S. Sutton and Andrew G. Barto, MIT Press, 2018.

http://incompleteideas.net/book/the-book-2nd.html

Relevant section: Chapter 3 (finite Markov decision processes) and Section 6.5 (Q-learning)

Why use it: Primary textbook support for the agent/environment distinction, expected return and the Q-learning update.

Learner task: After ACT-02, explain in your own words what the bootstrapped target includes, then compare it with the terminal-state case.

Limits: The book is mathematically deeper than this beginner guide. Start with the supplied worked calculation; do not treat the browser page as required activity content.

Offline alternative: The formula, both complete calculations and interpretation are included in ACT-02 and the chapter, so the task works offline.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R2 — Nand to Tetris — Project 5: Computer Architecture

Noam Nisan and Shimon Schocken / Nand to Tetris.

https://www.nand2tetris.org/project05

Relevant section: Project overview, proposed computer architecture and hardware simulator guidance

Why use it: A concrete educational computer architecture connects logic, CPU state and memory.

Learner task: After ACT-01, compare the purpose of an instruction address and a data-memory address with the project's computer model. Name one part the models share and one detail that differs.

Limits: The Hack computer is a different ISA; its instructions and registers must not be substituted directly into this guide's toy program. Software installation is optional and not required for the paper task.

Offline alternative: ACT-01 includes its full program, trace and expected output. ACT-03 provides a complete paper pipeline grid specification.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R3 — RFC 1122 — Requirements for Internet Hosts: Communication Layers

IETF / R. Braden, editor, 1989.

https://www.rfc-editor.org/rfc/rfc1122

Relevant section: Section 1.1.3 (Internet protocol suite) and the link, IP and transport sections

Why use it: An authoritative foundation for the layered Internet model used in Chapter 4.

Learner task: Draw the four teaching layers and place HTTP, TCP, IP and Ethernet/Wi-Fi by role. Explain why a large link capacity does not set an end-to-end response time.

Limits: This is a technical historical standards document; current protocols have later updates. Use it for layer roles, not as a complete modern security checklist.

Offline alternative: The chapter and ACT-04 contain the layer explanations and all delay inputs/results offline.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R4 — RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3

IETF / Eric Rescorla, 2018.

https://www.rfc-editor.org/rfc/rfc8446

Relevant section: Section 1 (Introduction) and Section 8 (0-RTT and anti-replay)

Why use it: Distinguishes transport confidentiality/integrity from application-level decisions, including replay considerations.

Learner task: After ACT-05, explain why authenticated transport alone does not decide whether an old command is still appropriate to execute.

Limits: The Python receiver is a validation demonstration, not a TLS implementation. Do not implement cryptography from this worksheet.

Offline alternative: ACT-05 includes the packet fields, state transitions and rejection explanations; no network service or account is needed.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R5 — Artificial Intelligence Risk Management Framework (AI RMF 1.0)

NIST, NIST AI 100-1, January 2023.

https://www.nist.gov/itl/ai-risk-management-framework

Relevant section: Framework functions Govern, Map, Measure and Manage; trustworthy AI characteristics

Why use it: Connects measured performance with context, impacts, monitoring and responsibility.

Learner task: For ACT-06, assign one concrete action to each framework function: identify an owner, describe the operating domain, measure a relevant failure and plan a response.

Limits: A voluntary risk-management framework is not certification that a product is safe, fair or legally compliant. Context and evidence still matter.

Offline alternative: ACT-06 supplies the design table and expected trade-off reasoning. A suitable four-action response is: Govern — name an accountable operator; Map — define the campus operating domain; Measure — test missed deadlines and uneven accuracy; Manage — maintain fallback and an incident response process.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

R6 — The Unicode Standard — technical introduction

The Unicode Consortium.

https://www.unicode.org/standard/principles.html

Relevant section: Unicode design principles and character encoding model

Why use it: Supports the difference between a code point and a variable-length encoding such as UTF-8.

Learner task: Explain why one character is not necessarily one byte, and why BCD digits and text digits need an agreed representation.

Limits: Full text processing includes combining characters and grapheme clusters beyond this chapter. A displayed symbol is not always one code point.

Offline alternative: Chapter 1 defines character sets, code points and encodings and includes the BCD/ordinary-binary comparison.

Access: Optional public web resource; no account required to read the cited page. Signed-out HTTP access was checked on 7 November 2026. Availability on every campus network is not guaranteed. All essential task instructions and feedback are supplied locally.

Responsible AI-use record

Neogpt/Codex assisted on 7 November 2026 with reconciling the guide into Markdown, developing activities and answers, drafting the Test Bank, generating PDFs/DOCX and verifying package structure. The user asked to follow rules/Stage2_All_Instructions.md. Calculations and supplied code outputs were checked locally; source/resource access and document checks are recorded in the submission evidence. These checks are not independent human peer review. Significant interaction excerpts belong in the project and personal logs.